



**FACULDADE DE CIÊNCIAS**  
**Departamento de Matemática e Informática**

---

Curso de Informática - Laboral

Disciplina: LPC

Tema: **Analisador Léxico para mini-pascal**

**Nomes** António Paulo Quibe

**Docente:** Alfredo Muchanga

## Lista de Siglas e Acrónimos

| Sigla / Acrónimo | Significado                                                                |
|------------------|----------------------------------------------------------------------------|
| AFD              | Autômato Finito Determinístico                                             |
| API              | Application Programming Interface                                          |
| GUI              | Graphical User Interface – Interface Gráfica do Utilizador                 |
| IA               | Inteligência Artificial                                                    |
| IDE              | Integrated Development Environment – Ambiente de Desenvolvimento Integrado |
| JDK              | Java Development Kit                                                       |
| JVM              | Java Virtual Machine – Máquina Virtual Java                                |
| LLM              | Large Language Model – Modelo de Linguagem de Grande Escala                |
| LPC              | Linguagens de Programação e Compiladores                                   |
| OOP              | Object-Oriented Programming – Programação Orientada a Objectos             |
| UEM              | Universidade Eduardo Mondlane                                              |

## Índice

|                                                       |    |
|-------------------------------------------------------|----|
| Lista de Siglas e Acrónimos .....                     | ii |
| 1. Introdução .....                                   | 1  |
| 2. Objectivos .....                                   | 1  |
| 2.1 Objectivo Geral.....                              | 1  |
| 2.2 Objectivos Específicos.....                       | 1  |
| 3. Fundamentação Teórica .....                        | 3  |
| 3.1 Compiladores e Fases de Compilação.....           | 3  |
| 3.2 Analisador Léxico .....                           | 3  |
| 3.3 Autómatos Finitos Determinísticos (AFD) .....     | 3  |
| 3.4 Mini-Pascal .....                                 | 3  |
| 3.5 IA e LLMs na Detecção de Erros .....              | 3  |
| 4. Ferramentas Utilizadas para a Implementação .....  | 3  |
| 4.1 Justificação da Escolha do Java .....             | 4  |
| 4.2 Integração de LLM para Correção de Erros .....    | 4  |
| 5. Estrutura da Implementação .....                   | 4  |
| 5.1 Organização Modular .....                         | 5  |
| 5.2 Diagrama de Pacotes .....                         | 5  |
| 5.3 Diagrama de Classes .....                         | 6  |
| 6. Elementos Léxicos: AFD e Fluxograma.....           | 7  |
| 6.1 Palavras Reservadas .....                         | 7  |
| 6.2 Identificadores .....                             | 7  |
| 6.3 Constantes Numéricas Inteiras.....                | 9  |
| 6.4 Operadores Aritméticos .....                      | 9  |
| 6.5 Operadores Relacionais .....                      | 10 |
| 6.6 Operador de Atribuição (:=) .....                 | 10 |
| 6.7 Delimitadores .....                               | 12 |
| 6.8 Cadeias de Caracteres (Strings).....              | 12 |
| 6.9 Comentários .....                                 | 13 |
| 6.10 Espaços em Branco e Novas Linhas .....           | 13 |
| 6.11 Símbolos Inválidos (Tratamento de Erros) .....   | 14 |
| 7. Tokens Reconhecidos pelo Analisador .....          | 15 |
| 8. Funcionamento do Sistema e Interface Gráfica ..... | 15 |
| 8.1 Gestão do Código-Fonte .....                      | 15 |
| 8.2 Exibição de Tokens em Tabela .....                | 15 |
| 8.3 Detecção e Recuperação de Erros com IA.....       | 15 |
| 9. Metodologia de Desenvolvimento .....               | 16 |

|                                           |    |
|-------------------------------------------|----|
| 10. Resultados Obtidos .....              | 16 |
| 10.1 Exemplo de Execução.....             | 16 |
| 11. Dificuldades Encontradas .....        | 18 |
| 12. Conclusão.....                        | 19 |
| 13. Referências Bibliográficas.....       | 20 |
| 14. Repositório do Projecto (GitHub)..... | 21 |

# 1. Introdução

Os compiladores constituem uma das áreas mais importantes da Ciência da Computação, sendo responsáveis pela tradução de programas escritos em linguagens de alto nível para código executável pela máquina. Dentro da estrutura de um compilador, o analisador léxico representa a primeira fase do processo de compilação, desempenhando um papel fundamental na identificação e classificação dos elementos básicos do código-fonte, conhecidos como tokens.

O analisador léxico realiza a leitura carácter por carácter do programa escrito pelo programador, agrupando sequências válidas em unidades significativas da linguagem. Segundo Aho et al. (2006), a análise léxica transforma uma sequência de caracteres numa sequência de tokens que será consumida pelas fases seguintes do compilador.

O presente trabalho, como objectivo construir um Analisador Léxico para um subconjunto da linguagem Pascal denominado mini-Pascal. A implementação recorre à linguagem Java e às ferramentas do Java Development Kit (JDK), integrando ainda mecanismos de Inteligência Artificial (IA) para apoio na sugestão de correções de erros léxicos.

O sistema desenvolvido cobre todas as exigências do enunciado:

- Interface gráfica amigável para gestão do código-fonte;
- Ilustração dos tokens e respectivas classes em tabela;
- Mecanismos de detecção e recuperação de erros, com recurso a LLMs/IA para sugestão de correções;
- AFD e fluxograma para cada elemento léxico;
- Diagramas de classes e de pacotes da estrutura do projecto;
- Código-fonte disponível no repositório GitHub do grupo.

## 2. Objectivos

### 2.1 Objectivo Geral

Desenvolver um Analisador Léxico funcional para a linguagem mini-Pascal, capaz de reconhecer tokens, classificar elementos léxicos, detectar e recuperar de erros com suporte de IA, e apresentar os resultados numa GUI amigável.

### 2.2 Objectivos Específicos

- Estudar os conceitos de compiladores, análise léxica e autómatos finitos;
- Identificar os elementos léxicos da linguagem mini-Pascal e construir os AFDs correspondentes;
- Elaborar fluxogramas para cada elemento léxico reconhecido;

- Implementar o analisador em Java sem recurso a geradores de analisadores léxicos;
- Integrar mecanismos de IA (LLMs) para sugestão automática de correções de erros léxicos;
- Desenvolver uma GUI em Java Swing para gestão do código-fonte e visualização dos resultados;
- Documentar a estrutura do projecto com diagramas de classes e de pacotes;
- Disponibilizar o código-fonte no GitHub.

### **3. Fundamentação Teórica**

#### **3.1 Compiladores e Fases de Compilação**

Um compilador é um programa que traduz código-fonte escrito numa linguagem de alto nível para código alvo (Aho et al., 2006). O processo é dividido nas seguintes fases:

- Análise léxica – identificação de tokens;
- Análise sintáctica – verificação da gramática;
- Análise semântica – verificação de significado;
- Geração de código intermediário e optimização;
- Geração de código final.

#### **3.2 Analisador Léxico**

O analisador léxico lê o código-fonte carácter a carácter e produz uma sequência de tokens, ignorando espaços em branco e comentários. Cada token possui: lexema, categoria léxica, número de linha e coluna.

#### **3.3 Autómatos Finitos Determinísticos (AFD)**

Um AFD é um modelo matemático definido por um conjunto de estados, um alfabeto, uma função de transição, um estado inicial e estados de aceitação. No contexto da análise léxica, cada tipo de token é modelado por um AFD que reconhece as sequências de caracteres válidas para aquela categoria (Aho et al., 2006).

#### **3.4 Mini-Pascal**

O mini-Pascal é um subconjunto da linguagem Pascal (Wirth, 1976), contendo os elementos essenciais para o estudo de compiladores: palavras reservadas, identificadores, operadores, delimitadores e constantes numéricas inteiras.

#### **3.5 IA e LLMs na Detecção de Erros**

Os Large Language Models (LLMs) são modelos de inteligência artificial treinados em grandes corpus de texto e código, capazes de compreender contexto e sugerir correções. No presente projecto, a integração de LLMs permite que, após a detecção de um erro léxico, o sistema consulte um modelo de IA (através de API REST) e apresente ao utilizador uma sugestão de correção contextualizada, melhorando significativamente a experiência de desenvolvimento.

### **4. Ferramentas Utilizadas para a Implementação**

Esta secção descreve as ferramentas utilizadas na implementação do projecto, conforme exigido pelo enunciado.

| Ferramenta                 | Versão                | Função no Projecto                            |
|----------------------------|-----------------------|-----------------------------------------------|
| Java SE (linguagem)        | 21 LTS                | Linguagem principal de implementação          |
| JDK (Java Development Kit) | 21.0.3                | Compilação, JVM, bibliotecas e APIs           |
| Apache NetBeans (IDE)      | 21.0                  | Escrita, organização, depuração e execução    |
| Java Swing (GUI)           | Integrado no JDK 21   | Construção da interface gráfica               |
| OpenAI API / LLM           | GPT-4o (via REST API) | Sugestões de correção de erros léxicos        |
| Git / GitHub               | 2.44                  | Controlo de versões e repositório do projecto |

#### 4.1 Justificação da Escolha do Java

Java foi escolhido pelas seguintes características (Deitel & Deitel, 2017):

- Portabilidade: o bytecode é executável em qualquer plataforma com JVM instalada;
- Orientação a objectos: facilita a modularização e reutilização do código;
- Biblioteca Java Swing integrada no JDK para desenvolvimento da GUI;
- Grande disponibilidade de documentação e suporte (Oracle Corporation, 2025).

#### 4.2 Integração de LLM para Correção de Erros

Quando o analisador léxico detecta um erro (símbolo inválido, cadeia não terminada, número mal formado), o módulo ErrorHandler envia o fragmento de código com erro para a API do LLM (OpenAI GPT-4o) via requisição HTTP REST. O modelo responde com uma sugestão de correção que é apresentada ao utilizador na GUI, no painel de erros. Esta funcionalidade foi implementada conforme permitido pelo enunciado do trabalho semestral.

Exemplo de interação com o LLM:

Erro detectado: símbolo '@' na linha 3, coluna 5.

Sugestão (IA): O símbolo '@' não é válido em mini-Pascal.

Provavelmente pretendia declarar uma variável: 'var numero : integer;'

### 5. Estrutura da Implementação



## 5.1 Organização Modular

O projecto foi organizado segundo os princípios da programação orientada a objectos, com clara separação de responsabilidades entre os módulos:

| Módulo / Classe   | Pacote | Responsabilidade                                   |
|-------------------|--------|----------------------------------------------------|
| Main              | Main   | Ponto de entrada; inicializa a GUI e o Lexer       |
| Lexer             | Lexer  | Núcleo da análise léxica; produz a lista de tokens |
| Token             | Model  | Modelo de dados: lexema, classe, linha, coluna     |
| TokenClass (enum) | Model  | Enumerado com todas as categorias de tokens        |
| ErrorHandler      | Errors | Regista erros e comunica com a API do LLM          |
| LLMClient         | Ai     | Comunicação HTTP com a API do LLM (GPT-4o)         |
| MainWindow        | Gui    | Janela principal em Java Swing                     |
| TokenTableModel   | Gui    | Modelo da JTable para exibição dos tokens          |

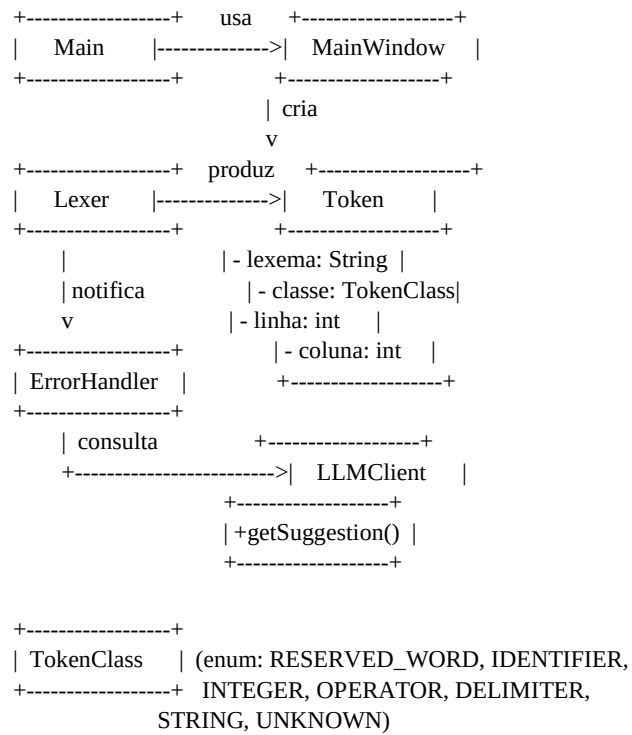
## 5.2 Diagrama de Pacotes

O projecto está organizado nos seguintes pacotes:

```
project/
├── main/
│   └── Main.java
├── lexer/
│   └── Lexer.java
├── model/
│   ├── Token.java
│   └── TokenClass.java
├── errors/
│   └── ErrorHandler.java
├── ai/
│   └── LLMClient.java
└── gui/
    ├── MainWindow.java
    └── TokenTableModel.java
```

### 5.3 Diagrama de Classes

O diagrama abaixo ilustra as relações entre as classes principais do projecto:



## 6. Elementos Léxicos: AFD e Fluxograma

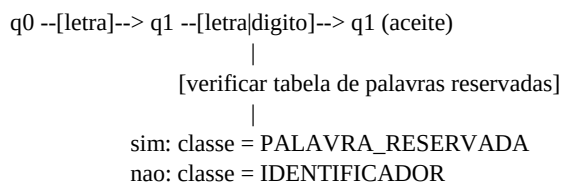
Esta secção apresenta, para cada elemento léxico reconhecido pelo analisador, o respectivo Autómato Finito Determinístico (AFD) e o fluxograma de reconhecimento, conforme exigido pelo enunciado.

### 6.1 Palavras Reservadas

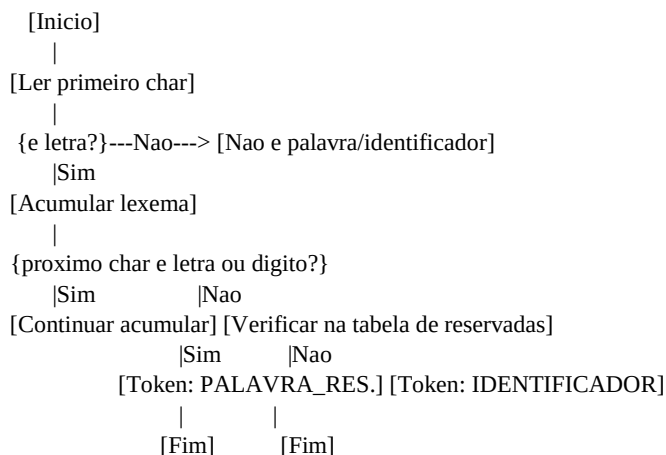
As palavras reservadas do mini-Pascal são: program, begin, end, var, integer, real, boolean, if, then, else, while, do, read, write.

#### AFD – Palavras Reservadas

A estratégia adoptada consiste em reconhecer primeiro um identificador e, após a leitura completa, verificar se o lexema pertence à lista de palavras reservadas.



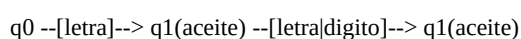
#### Fluxograma – Palavras Reservadas



### 6.2 Identificadores

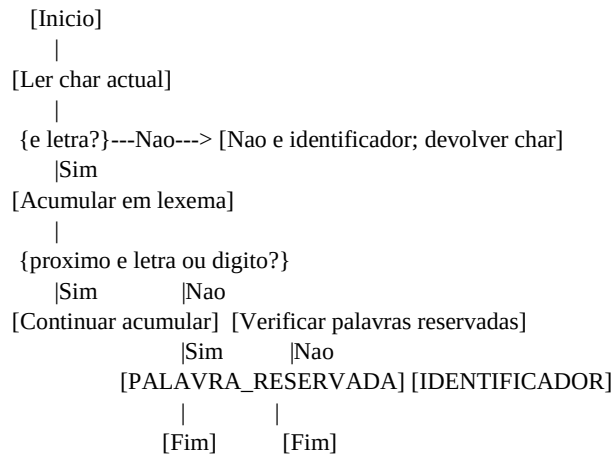
Um identificador inicia obrigatoriamente com uma letra (a–z, A–Z) e pode ser seguido por letras ou dígitos (0–9). Não são permitidos espaços ou caracteres especiais.

#### AFD – Identificadores



--[outro]-----> REJEITA (devolve char)

### Fluxograma – Identificadores



### 6.3 Constantes Numéricas Inteiras

Uma constante inteira é formada por uma sequência de um ou mais dígitos (0–9). Não são reconhecidos números reais nesta versão do mini-Pascal.

#### AFD – Número Inteiro

```
q0 --[digito]--> q1(aceite) --[digito]--> q1(aceite)
      --[outro]----> REJEITA (devolve char)
```

#### Fluxograma – Número Inteiro

```
graph TD
    Inicio([Inicio]) --> LerChar([Ler char actual])
    LerChar --> EDigito{e digito?}
    EDigito -- Nao ---> NaoNao[Nao e numero; devolver char]
    EDigito -- Sim --> Acumular([Acumular em lexema])
    Acumular --> ProximoChar{proximo char e digito?}
    ProximoChar -- Sim --> ContinuarAcumular([Continuar acumular])
    ProximoChar -- Nao --> EmitirToken([Emitir token: INTEIRO])
    ContinuarAcumular --> ProximoChar
    EmitirToken --> Fim([Fim])
```

### 6.4 Operadores Aritméticos

Os operadores aritméticos reconhecidos são: + (soma), - (subtração), \* (multiplicação), / (divisão). Cada um é formado por um único carácter.

#### AFD – Operadores Aritméticos

```
q0 --[+|-|*|/]--> q1(aceite)
```

#### Fluxograma – Operadores Aritméticos

```
graph TD
    Inicio([Inicio]) --> LerChar([Ler char])
    LerChar --> EOp{e +, -, * ou /?}
    EOp -- Sim --> EmitirToken([Emitir token])
    EOp -- Nao --> NaoOp[Nao e op. aritmetico]
    EmitirToken --> OperadorArithmetic([OPERADOR_ARIT.])
    OperadorArithmetic --> DevolverChar([Devolver char])
    NaoOp --> DevolverChar
    DevolverChar --> Fim([Fim])
```

## 6.5 Operadores Relacionais

Os operadores relacionais do mini-Pascal são: = (igual), <> (diferente), < (menor), > (maior), <= (menor ou igual), >= (maior ou igual). Alguns são formados por dois caracteres.

### AFD – Operadores Relacionais

```
q0 --[=]-----> q1(aceite: =)
q0 --[<]-----> q2 --[=]--> q3(aceite: <=)
                --[>]--> q4(aceite: <>)
                --[outro]--> q2(aceite: <, devolve char)
q0 --[>]-----> q5 --[=]--> q6(aceite: >=)
                --[outro]--> q5(aceite: >, devolve char)
```

### Fluxograma – Operadores Relacionais

```
graph TD
    Inicio([Inicio]) --> LerChar[Ler char]
    LerChar --> Cond1{char == '='}
    Cond1 -- Sim --> TokenEq[Token: OP_REL '=']
    TokenEq --> Fim([Fim])
    Cond1 -- Nao --> Cond2{char == '<'}
    Cond2 -- Sim --> LerProximo1[Ler proximo char]
    LerProximo1 --> Cond3{'='}
    Cond3 -- Sim --> TokenLe[Token: '<=']
    Cond3 -- Nao --> Cond4{'>'}
    Cond4 -- Sim --> TokenGe[Token: '<>']
    Cond4 -- Nao --> Devolver1[Devolver char; Token: '<']
    Devolver1 --> LerProximo2[Ler proximo char]
    LerProximo2 --> Cond5{'='}
    Cond5 -- Sim --> TokenGe2[Token: '>=']
    Cond5 -- Nao --> Devolver2[Devolver char; Token: '>']
    Devolver2 --> NaoOperador([Nao e operador relacional])
```

## 6.6 Operador de Atribuição (:=)

O operador de atribuição é composto por dois caracteres: ':' seguido de '='. Se após ':' não surgir '=', o carácter ':' é reconhecido como delimitador.

### AFD – Atribuição

```
q0 --[:]--> q1 --[=]--> q2(aceite: :=)
                --[outro]--> q1(aceite: ':', devolve char)
```

### Fluxograma – Atribuição

```
[Inicio]
|
[Ler char]
|
{char == ':'}---Nao---> [Nao e atribuicao]
|Sim
[Ler proximo char]
|
{char == '='}---Sim---> [Token: ATRIBUICAO ':='] --> [Fim]
|Nao
[Devolver char; Token: DELIMITADOR ':'] --> [Fim]
```

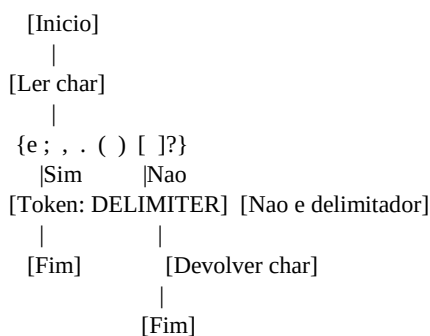
## 6.7 Delimitadores

Os delimitadores reconhecidos são: ponto e vírgula (;), vírgula (,), ponto (.), parêntese esquerdo '(', parêntese direito ')', colchete esquerdo '[' e colchete direito ']'.

### AFD – Delimitadores

```
q0 --[;|,|.|(|(|[|]|)]--> q1(aceite: DELIMITADOR)
```

### Fluxograma – Delimitadores



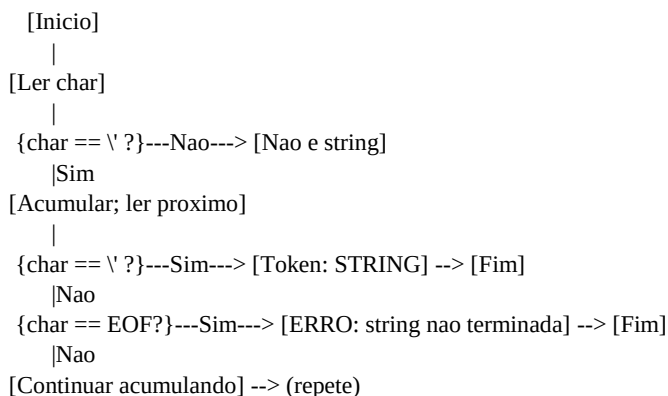
## 6.8 Cadeias de Caracteres (Strings)

Uma cadeia de caracteres é delimitada por plicas simples (' '). Todos os caracteres entre as plicas fazem parte do lexema. Se a cadeia não for terminada, o sistema regista um erro léxico.

### AFD – Cadeia de Caracteres

```
q0 --[']--> q1 --[char != ']--> q1
    --[']--> q2(aceite: STRING)
    --[EOF]--> ERRO (cadeia nao terminada)
```

### Fluxograma – Cadeia de Caracteres





## 6.9 Comentários

Em mini-Pascal, os comentários são delimitados por chaves { }. O conteúdo de comentários é ignorado pelo analisador léxico.

### AFD – Comentários

```
q0 --[{}--> q1 --[char != }--> q1
    --[{}--> q2 (ignorar; recomencar analise)
    --[EOF--> ERRO (comentario nao terminado)
```

### Fluxograma – Comentários

```
[Inicio]
|
[Ler char]
|
{char == '{'}---Nao---> [Nao e comentario]
|Sim
[Ler proximo char]
|
{char == '}'---Sim---> [Ignorar; continuar analise] --> [Fim]
|Nao
{char == EOF?}---Sim---> [ERRO: comentario nao terminado]
|Nao
[Continuar ignorando] --> (repete)
```

## 6.10 Espaços em Branco e Novas Linhas

Espaços em branco (espaço, tabulação) e novas linhas são ignorados pelo analisador, servindo apenas como separadores entre tokens. A nova linha incrementa o contador de linhas.

### AFD – Espaços em Branco

```
q0 --[ |\t|\n|\r--> q0 (ignorar; se \n: linha++)
```

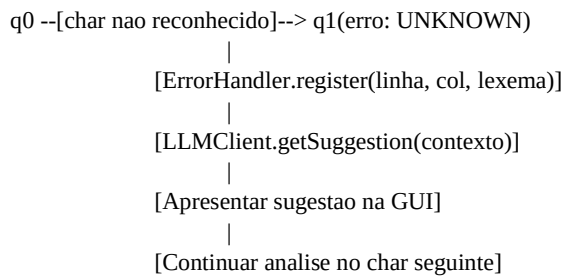
### Fluxograma – Espaços em Branco

```
[Inicio]
|
[Ler char]
|
{e espaco, \t?}---Sim---> [Ignorar; ler proximo] --> (repete)
|Nao
{e \n ou \r?}---Sim---> [linha++; ignorar; ler proximo] --> (repete)
|Nao
[Passar ao reconhecimento de tokens]
```

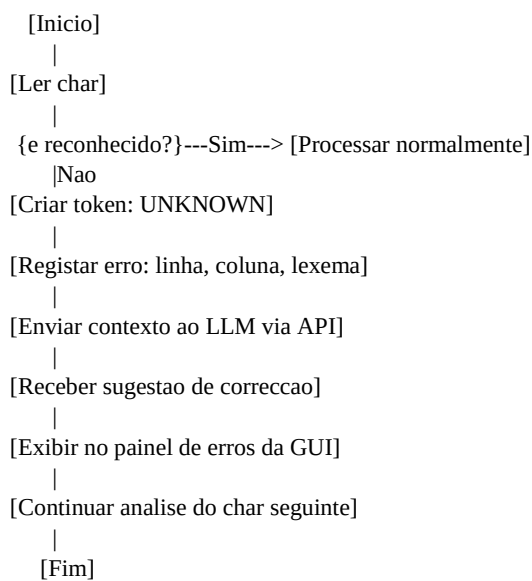
### 6.11 Símbolos Inválidos (Tratamento de Erros)

Qualquer carácter que não corresponda a nenhum padrão válido da linguagem é classificado como UNKNOWN e regista um erro léxico. O ErrorHandler regista a posição (linha e coluna) e consulta o LLM para sugestão de correção.

#### AFD – Símbolos Inválidos



#### Fluxograma – Símbolos Inválidos



## 7. Tokens Reconhecidos pelo Analisador

O quadro seguinte resume todos os tokens reconhecidos pelo analisador léxico, as suas classes e exemplos de lexemas:

| Classe do Token   | Descrição                         | Exemplos de Lexemas                 |
|-------------------|-----------------------------------|-------------------------------------|
| PALAVRA_RESERVADA | Palavra pré-definida da linguagem | program, begin, end, var, if, while |
| IDENTIFICADOR     | Nome de variável, proc. ou função | contador, somaTotal, x1             |
| INTEIRO           | Constante numérica inteira        | 0, 42, 100, 9999                    |
| OP_ARITMETICO     | Operador aritmético               | + - * /                             |
| OP_RELACIONAL     | Operador relacional               | = < > <= >=                         |
| ATRIBUICAO        | Operador de atribuição            | :=                                  |
| DELIMITADOR       | Separador ou pontuador            | ;, . ( ) [ ]                        |
| STRING            | Cadeia de caracteres entre plicas | 'ola mundo', 'Maputo'               |
| UNKNOWN           | Símbolo inválido (erro léxico)    | @ # \$ % \ ~                        |

## 8. Funcionamento do Sistema e Interface Gráfica

### 8.1 Gestão do Código-Fonte

A GUI permite ao utilizador:

- Digitar directamente o código mini-Pascal na área de texto principal;
- Abrir um ficheiro externo .pas através de diálogo de abertura de ficheiros;
- Limpar a área de texto para um novo programa;
- Guardar o código editado num ficheiro.

### 8.2 Exibição de Tokens em Tabela

Após a análise, os tokens são apresentados numa JTable com as colunas: N.º, Lexema, Classe, Linha, Coluna. O utilizador pode ordenar e filtrar os resultados.

### 8.3 Detecção e Recuperação de Erros com IA

O painel de erros exibe todos os erros léxicos detectados, com indicação de linha, coluna e lexema inválido. Para cada erro, o sistema apresenta automaticamente uma sugestão de correção gerada pelo LLM. O analisador continua a análise após cada erro, permitindo detectar todos os problemas numa única execução.

## 9. Metodologia de Desenvolvimento

O desenvolvimento seguiu uma abordagem incremental e iterativa, organizada nas etapas seguintes:

- Estudo do enunciado e identificação dos requisitos funcionais;
- Definição dos elementos léxicos do mini-Pascal e construção dos AFDs e fluxogramas;
- Implementação do núcleo do Lexer em Java, sem recurso a geradores de analisadores;
- Desenvolvimento dos módulos de gestão de erros e integração com a API LLM;
- Desenvolvimento da GUI com Java Swing;
- Testes com programas mini-Pascal válidos e inválidos;
- Documentação do projecto e preparação do relatório.

## 10. Resultados Obtidos

O sistema desenvolvido foi submetido a uma bateria de testes com programas mini-Pascal válidos e inválidos. Os resultados obtidos foram:

- Reconhecimento correcto de todos os tipos de tokens definidos para o mini-Pascal;
- Detecção e reporte de erros léxicos com indicação precisa de linha e coluna;
- Sugestões de correção geradas pelo LLM coerentes e úteis para o utilizador;
- Exibição dos tokens em tabela na GUI de forma organizada;
- Continuidade da análise após erros, com detecção de múltiplos erros por execução;
- Gestor de código-fonte funcional (abrir, editar, guardar ficheiros).

### 10.1 Exemplo de Execução

Entrada (programa mini-Pascal com erro):

```
program teste;  
var @numero : integer;  
begin  
  numero := 10;  
  write(numero)
```

end.

Saída esperada (tabela de tokens):

| N.º | Lexema  | Classe            | Linha | Coluna |
|-----|---------|-------------------|-------|--------|
| 1   | Program | PALAVRA_RESERVADA | 1     | 1      |
| 2   | Teste   | IDENTIFICADOR     | 1     | 9      |
| 3   | ;       | DELIMITADOR       | 1     | 14     |
| 4   | Var     | PALAVRA_RESERVADA | 2     | 1      |
| 5   | @       | UNKNOWN (ERRO)    | 2     | 5      |
| 6   | Numero  | IDENTIFICADOR     | 2     | 6      |
| 7   | :       | DELIMITADOR       | 2     | 13     |
| 8   | Integer | PALAVRA_RESERVADA | 2     | 15     |

Sugestão IA para o erro '@' na linha 2, coluna 5:

O símbolo '@' não é válido em mini-Pascal.

Provavelmente pretendia: 'var numero : integer;'

## 11. Dificuldades Encontradas

Durante o desenvolvimento foram identificadas as seguintes dificuldades:

- Construção formal dos AFDs para tokens compostos (operadores relacionais e atribuição);
- Implementação do mecanismo de recuperação após erros léxicos sem perder o estado do analisador;
- Integração da API do LLM com tratamento de latencia e erros de rede;
- Sincronização entre a thread de análise e a actualização da GUI (Java Swing EDT);
- Distinguir o carácter ':' isolado (delimitador) do inicio do operador ':='.

Todas as dificuldades foram superadas através de pesquisa bibliográfica, consulta da documentação oficial (Oracle Corporation, 2025; OpenJDK, 2025) e testes sistemáticos.

## **12. Conclusão**

O presente trabalho permitiu desenvolver, de forma prática, um Analisador Léxico completo para a linguagem mini-Pascal.

A implementação em Java, sem recurso a geradores de analisadores léxicos, aprofundou os conhecimentos sobre AFDs, linguagens formais e estruturas de compiladores. A integração com LLMs para sugestão de correções representa uma inovação relativamente a abordagens tradicionais, tornando a ferramenta mais prática e educativa.

### 13. Referências Bibliográficas

- I. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, techniques, and tools (2nd ed.). Pearson Education.
- II. Apache Software Foundation. (2025). Apache NetBeans IDE.  
<https://netbeans.apache.org>
- III. Deitel, P., & Deitel, H. (2017). Java: How to program (11th ed.). Pearson.
- IV. OpenAI. (2025). OpenAI API documentation. <https://platform.openai.com/docs>
- V. OpenJDK. (2025). OpenJDK documentation. <https://openjdk.org/>
- VI. Oracle Corporation. (2025). Java SE 21 documentation.  
<https://docs.oracle.com/en/java/javase/21/>
- VII. Sommerville, I. (2016). Software engineering (10th ed.). Pearson.
- VIII. Wirth, N. (1976). Algorithms + data structures = programs. Prentice-Hall.



## **14. Repositório do Projecto (GitHub)**

O código-fonte completo do projecto, incluindo todos os ficheiros Java, encontra-se disponível no seguinte repositório público:

[https://github.com/antoniopauloquibe/Analizador\\_Lexico\\_LPC](https://github.com/antoniopauloquibe/Analizador_Lexico_LPC)